

Neural networks: a calculus perspective

David Puelz

The University of Austin



What we'll see today

- Neural networks as **compositions of functions**
- The **chain rule** is the engine behind training
- **Gradient descent** minimizes a loss function
- **Backpropagation**: efficient gradient computation for many parameters
- All high-level—no heavy derivations

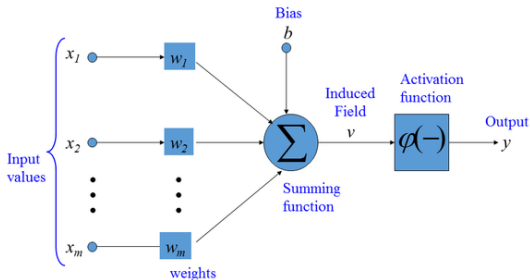


Neural networks in one sentence

- Neural nets = fancy regression models built from many simple pieces
- Each “neuron” computes: linear combination $\rightarrow$ activation function
- We choose weights and biases to minimize prediction error
- The math that makes this work: derivatives and the chain rule

Basic concepts: neurons

- Basic building blocks of neural networks
- Each neuron is a **composition** of two steps:
 - 1. Linear combination: $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
 - 2. Activation: $y = \phi(z)$ (e.g., sigmoid, ReLU)
- Weights w_i and bias b are **parameters** we learn





Layers and composition

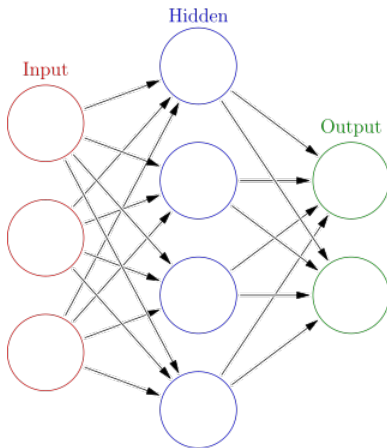
- **Input layer:** receives features $x_1, \dots, x_n$
- **Hidden layers:** each neuron applies $z \mapsto \phi(z)$ to a linear combination of inputs
- **Output layer:** produces predictions $\hat{y}$
- The full network is a **composition of many functions:**

$$\hat{y} = f_L(f_{L-1}(\cdots f_1(\mathbf{x}) \cdots))$$

- Calculus question: how does changing a weight w affect the final output?



Network structure (visual)





Loss function: what we minimize

- We have data $(\mathbf{x}^{(i)}, y^{(i)})$ and predictions $\hat{y}^{(i)}$
- **Mean squared error (MSE):**

$$L(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$$

- L depends on **all** weights and biases through the composition
- Goal: find parameters that **minimize** L
- This is a **multivariable optimization** problem



Gradient descent: the big idea

- We can't usually solve $\nabla L = \mathbf{0}$ in closed form
- **Gradient descent**: take small steps in the direction that **decreases** L

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \nabla L(\mathbf{w}^{(k)})$$

- α = learning rate (step size)
- The gradient ∇L points **uphill**; $-\nabla L$ points **downhill**
- Key: we need $\frac{\partial L}{\partial w}$ for every weight w



The chain rule (review)

- For $y = f(g(x))$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} \quad \text{where } u = g(x)$$

- For longer chains $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

- Each link in the chain contributes a factor
- In neural nets: $L \rightarrow \hat{y} \rightarrow z \rightarrow w$ (and similar for every parameter)

Single neuron example: the chain

- One neuron: $\hat{y} = \sigma(wx + b)$, loss $L = \frac{1}{2}(\hat{y} - y)^2$
- Chain: L depends on $\hat{y}$, which depends on $z = wx + b$, which depends on w and b

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- **Step 1:** $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$ (error)
- **Step 2:** $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$ (sigmoid derivative)
- **Step 3:** $\frac{\partial z}{\partial w} = x$
- So: $\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot x$



Backpropagation: what is it?

- In a deep network, we have **thousands** of weights
- Computing $\frac{\partial L}{\partial w}$ for each one by hand is impractical
- **Backpropagation**: an efficient algorithm to compute all gradients at once
- It uses the chain rule **layer by layer**, propagating errors **backward**
- Same calculus; clever organization



Forward vs. backward pass

- **Forward propagation:** input $\rightarrow$ layer 1 $\rightarrow$ layer 2 $\rightarrow \dots \rightarrow$ output
- We compute activations $a^{(1)}, a^{(2)}, \dots$ and the loss L
- **Backpropagation:** output $\rightarrow$ layer $L \rightarrow$ layer $L-1 \rightarrow \dots \rightarrow$ input
- We compute “how much each layer contributed to the error”
- At each layer, we use the chain rule to pass gradients to the previous layer



Backpropagation: high-level steps

1. **Forward pass:** compute $\hat{y}$ and L (store intermediate values)
2. **Output layer:** compute $\frac{\partial L}{\partial \hat{y}}$ (the error signal)
3. **Backward pass:** for each layer l from output to input:
 - Use the chain rule: “gradient from next layer” $\times$ “local derivative”
 - Local derivative = derivative of activation w.r.t. its input
 - Propagate gradient to the previous layer
4. **Update:** each weight gets its gradient; apply gradient descent



Why “back” propagation?

- The chain rule multiplies derivatives from the output toward the input
- We start at the loss L and work backward through the network
- Each layer receives a “gradient signal” from the layer after it
- We combine that with our local derivative (activation, weights)
- Result: efficient computation—we reuse intermediate values from the forward pass



The chain rule in backpropagation

- For a composite function $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

- In a network: L depends on layer L 's output, which depends on layer $L-1, \dots$

- Backprop computes $\frac{\partial L}{\partial (\text{layer } l \text{ output})}$ for each layer

- Then: $\frac{\partial L}{\partial w_{ij}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}}$

- Each factor is a **local** derivative we can compute easily



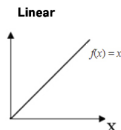
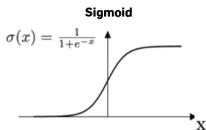
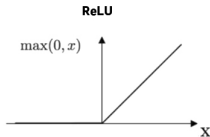
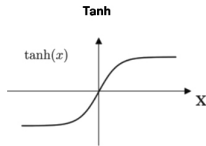
Training loop (conceptual)

- **Initialize** weights and biases (randomly)
- **Repeat** for many iterations:
 - Forward pass: compute $\hat{y}$ and L
 - Backward pass: compute ∇L for all parameters
 - Update: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L$
- Stop when L is small enough or after a fixed number of steps
- This is **gradient descent** applied to the loss surface



Activation functions (brief)

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$; derivative $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- **ReLU:** $f(x) = \max(0, x)$; derivative is 0 or 1
- **Tanh:** $\tanh(x)$; similar to sigmoid but centered at 0
- These derivatives appear in the chain rule during backprop





Summary: calculus in neural networks

- **Composition:** a network is $f_L \circ f_{L-1} \circ \dots \circ f_1$
- **Loss:** L measures prediction error; we minimize it
- **Gradient descent:** $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L$
- **Chain rule:** $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$ (and similar for every parameter)
- **Backpropagation:** efficient way to compute all gradients using the chain rule layer by layer



- Neural networks are **optimization problems**: minimize loss over many parameters
- The **derivative** (and chain rule) tells us how to improve
- **Backpropagation** is not new math—it's the chain rule, organized for efficiency
- Everything you've learned about gradients and minimization applies here